

Can LLMs Write Code That Writes Code?

Software development with AI/LLM support has moved from experiment to normal engineering practice. Many projects have already been built with this approach. The question "Can an LLM help generate code?" is no longer theoretical. The practical answer is yes, it can.

But that answer opens a more interesting question.

If an LLM can help generate code, can it also help generate code that generates code?

Content

1. The problem
2. What knime2py actually does
3. The development setup
4. The central claim
5. Important note: Generators multiply mistakes
6. Where LLMs perform best
7. Where LLMs struggle
8. Verification is the real constraint
9. Applications
10. Conclusion

That was the real question behind the knime2py (<https://github.com/vitalii-kaplan/knime2py>) project. The goal was to convert KNIME (<https://www.knime.com/>) workflows into readable Python scripts and notebooks. The LLM was not used only to help write Python code directly. It was used to help build a tool that writes Python code. That is a different engineering problem.

In this article, I want to share my experience of developing a code generator with the help of an LLM. I will show the process, the strong parts, the pitfalls, and the limitations. I hope this experience will help readers apply the same lessons in their own projects. The question is not whether an LLM can produce code that looks like a code generator. It can. The harder question is whether that generator can preserve enough meaning from the original system that engineers and analysts can trust, test, inspect, and repair the output.

1. The Problem

In my practice, I work with ML projects written in Python and with KNIME workflows. Both approaches have strengths and weaknesses. KNIME gives a very strict workflow structure and a clear visual data flow. Python is more flexible in terms of project structure and developer expression. It can also be inspected, tested, reviewed, versioned, and modified directly by end users.

At some point I asked myself whether it was possible to create a bridge between these two approaches.

If a KNIME workflow already has a strict structure and explicit data flow, can we convert that structure into Python code? Python is flexible enough to adopt many different shapes, so maybe it can adopt the structure of a KNIME workflow too.

To explore this, I started an open-source project called `knime2py`. It had two aims. The first was to check whether an LLM could be used to help generate a code generator. The second was to create a converter suitable for end users, not only for research experiments.

The original task behind `knime2py` could be stated in one question: Given a KNIME workflow, can we create Python code from it?

The tool has to parse a KNIME workflow, reconstruct nodes and connections, understand node settings, preserve execution order, and emit runnable Python code. It also has to make the generated code readable enough that a person can inspect it later. This is not ordinary LLM code generation. If I ask an LLM to write a function, the function is the thing I evaluate. If I ask an LLM to help write `knime2py`, the result is one level removed. I evaluate the generator by looking at the code it produces, running that code, and comparing its behavior with the original KNIME workflow.

2. What `knime2py` Actually Does

`knime2py` is a KNIME-to-Python exporter. It parses a workflow, reconstructs its nodes and connections, splits isolated connected graphs, and emits Python workbooks as scripts or notebooks. It also writes graph descriptions as JSON and Graphviz DOT files.

The process has two main phases:

First, `knime2py` creates graphs of connected nodes. If a workflow has separate isolated components, each component becomes a separate output set.

Second, it traverses each graph and creates a linear sequence of Python code for the nodes in that graph. The generated Python uses shared context variables to pass data and parameters from node to node.

The generated script has a consolidated import block, separate sections for workflow nodes, and a shared context dictionary for passing intermediate tables and parameters. A single run can also produce a JSON graph file, a DOT graph file, a Python script, and a Jupyter notebook. So the generator is not producing an isolated snippet. It is producing both executable Python and a structured representation of the original workflow in another programming environment.

3. The Development Setup

Feel free to skip this paragraph if you don't need technical details. The project was developed with a mixture of browser-based ChatGPT discussions and Codex in VS Code. The browser chats were split by activity: development, documentation, deployment, UI, tests, and RAG. Each chat had its own history and context. Browser chats were useful for discussion, planning, and architecture. Codex was used for direct work with code, implementation of decisions, and tests. The project also used `README.md` and `AGENTS.md`. `AGENTS.md` acted as long-term memory for Codex. A code generator is not a one-prompt project. It needs persistent rules, naming conventions, design decisions, examples, and reminders about what the project is trying to preserve. The repository also had scripts with examples of runs and parameters, YAML files for project structure and GitHub builders, and a Makefile for routine operations. This setup is not incidental. When an LLM helps build a tool like this, the project needs stable

context around it. Otherwise, the model can produce locally plausible code that does not fit the larger system.

4. The Central Claim

LLMs can help write code generators, but only under engineering control. The hard part is not producing code text. The hard part is making the generator preserve workflow semantics, handle node-specific behavior, expose unsupported parts clearly, and produce output that can be tested against the original system. In `knime2py`, the goal was never to implement all of KNIME. KNIME is a large product with years of development behind it. Reproducing all of it in one open-source project is not realistic. The practical goal was narrower: generate Python code that is as close as possible to the original internal KNIME workflow, while keeping the output human-readable and easy to inspect. If a segment is not implemented, the user should be able to find it and fix it manually.

5. Important Note: Generators Multiply Mistakes

A bug in a normal script is local. It may still be serious, but it lives in one place. A bug in a generator is different. The generator repeats the mistake every time it sees the same pattern. In `knime2py`, a node exporter is not a one-off implementation. It becomes the rule for translating that type of KNIME node into Python. If that rule misunderstands a parameter, mishandles an input port, or writes the wrong context key, the mistake can spread across every generated workbook that uses that node pattern. The generated Python may still look clean. It may have imports, comments, node sections, and a consistent structure. But consistency is not correctness. A generator can be consistently wrong. This is why review has to include both levels: the generator and representative generated outputs. It is not enough to say that the generator source looks reasonable. The emitted Python has to run, and its behavior has to be compared with the original workflow.

6. Where LLMs Perform Best

LLMs perform best when the pattern is explicit. Data-processing nodes were the most straightforward case. These nodes take tables as input and return tables as output. For each node, the LLM could use the node's `settings.xml` from a KNIME project. That file gave parameter names and values. The prompt could describe the transformation that the Python code should perform based on the XML configuration. With tests based on multiple `settings.xml` examples for the same node, the generator could be tuned for different node configurations and states. This is a strong LLM-assisted task. The input is explicit. The expected transformation can be described. The output can be tested. For these nodes, tests can check that the Python result is exactly the same as the KNIME workflow result on the same data. Text reading and writing nodes were also manageable. They could be implemented and tested directly by checking write-read behavior and strict equality of results. These are the cases where LLM-assisted generator development makes the most sense. The rules are visible enough to encode, and the outputs are deterministic enough to verify.

7. Where LLMs Struggle

7.1 Machine Learning Nodes and Hidden Parameters

The harder cases start when the rules are not fully visible. Machine learning nodes are a good example. In KNIME, many algorithms are represented as pairs of nodes: a Learner and a Predictor. For example, a Logistic Regression Learner creates a model, and a Logistic Regression Predictor applies it. The same pattern exists for Decision Trees, Random Forest, Gradient Boosted Trees, Neural Networks, and other algorithms. Predictor nodes are usually more straightforward: they receive a trained model and a table, then apply the model to the table. Learner nodes are harder because they create the model, and this is where hidden parameters and implementation details matter most. The LLM could be given the settings.xml file, the algorithm name, and references to already implemented algorithms of the same class. The main problem with ML nodes was not explaining the desired code shape. The main problem was hidden KNIME parameters and implementation differences. The KNIME UI exposes main meta-parameters, and those appear in settings.xml, but it does not expose every internal detail of the algorithm. Python gives more direct control over the process, especially when using pandas and scikit-learn. But this also means that a Python implementation with the same algorithm name is not necessarily the same algorithm in practice. KNIME's understanding of a neural network can differ from Python, pandas, and scikit-learn. As a result, applying the KNIME workflow and the generated Python script to the same data can produce different results.

7.2 Randomness and Reproducibility

Randomness creates a similar problem. Even with the same seed, KNIME Java libraries and Python libraries can generate different exact results. In these cases, strict equality is the wrong test. The project has to compare distributions and statistical measures, such as confusion matrices, instead of expecting identical rows or identical predictions.

7.3 Serialization Boundaries

Serialization is another hard boundary. Some KNIME nodes serialize blobs with undocumented internal structure, possibly through Java serialization. Without a detailed description of the Java classes behind those serialized objects, cross-serialization from KNIME to Python and back is not realistic. Python serialization can be supported inside Python, but Python cannot reliably read an object serialized by KNIME, and KNIME cannot reliably read an object serialized by Python.

7.4 UI Nodes

UI nodes are also not a direct translation. KNIME can expose workflow parameters through an interactive UI, especially in cloud usage. A Python script does not have the same interaction model. The practical compromise is to use default selections and create a special section near the beginning of the generated script where parameters can be tuned.

7.5 Flow-Control Nodes

Flow-control nodes introduce another class of complexity. KNIME workflows can contain loops. There are nodes for loop start, condition check, and loop end. Nodes between them are internal to the loop and use loop parameters. This means the generator cannot treat every

node as a purely local translation. It has to create an additional Python block and pass loop parameters into nodes inside the loop.

8. Verification Is the Real Constraint

The useful question is not: can the LLM produce code quickly?

The useful question is: can we prove that the generator behaves correctly enough?

knime2py uses both unit tests and functional tests. The project has about 85 percent test coverage. Unit tests cover main functionality and node behavior. Functional tests compare the result of data processing in KNIME with the result of generated Python code applied to the same data. The test data includes the KNIME workflow, original data, and expected result data. Python code for each node is generated and run at least once in functional tests. The model may help write the implementation, but the implementation has to survive execution and comparison.

Different node types need different test standards.

For deterministic data-processing nodes, strict equality is a reasonable expectation. The Python output should match the KNIME output on the same data.

For ML nodes and nodes with randomness, exact equality may be misleading or impossible. In those cases, the tests should check distributions, class proportions, split sizes, confusion matrices, and other statistical properties.

The point is not to force one verification method everywhere. The point is to make verification explicit.

9. Applications

If you have a KNIME workflow, you can try knime2py. It is open source, so users can clone the repository and run it directly. It also has releases as Docker images, PEX files, and EXE files. The Docker image was downloaded more than 700 times, so many users are comfortable running it from the command line. But not every KNIME user is comfortable with that. For that reason, a custom KNIME node UI (<https://vitalii-kaplan.github.io/knime2py/ui/>) was created. It requires a PEX or EXE package of knime2py.

In addition, a web interface was created at k2pweb.org (<https://k2pweb.org/>). During the first two months, more than 1,500 nodes were requested for export to Python through that interface. The most serious issues have been fixed, and the platform is stable.

10. Conclusion

My experience with knime2py is that LLMs can help build code generators, but only when the project gives them enough structure. The source format must be explicit, the target code must be defined, and verification must be part of the design from the beginning.

The main lesson is not that LLMs can replace engineering judgment. They cannot. The useful lesson is narrower and more practical: when the rules are clear and the outputs can be tested, LLMs can help turn those rules into working tools. When the rules are hidden, the hard work is still making them visible.

The next steps are practical. The first is to increase node coverage and support more KNIME nodes. The second is to research the difference between results produced by KNIME ML workflows and Python ML workflows generated from them. The third is to research generation of Python code with DuckDB and Polars instead of pandas. Each of these directions is still about the same core question: how much of the original workflow can be made explicit, translated, and verified?